

実験ログ: 見た目 (Karma・材質・テクスチャ)

2026-09-24 | 収録 12 件

Houdini 研究部で、hython で実際に測った実験の記録。数字はどれも測った値。

収録している実験

- ・ Karma でのレンダリング — ノイズはサンプル数で本当に減るのか
- ・ 浸食のレイヤーで地形を塗り分ける — 色が地形の形と対応しているか
- ・ 塗り分けた地形を Karma で出す — 見立ては外れ、白飛びが本命だった
- ・ Copernicus でテクスチャを作る — 細かさは解像度をどう食うのか
- ・ 作ったテクスチャを地形に貼る — 貼れたかどうかを画の数字で判定する
- ・ 凸凹の付け方3通り — 輪郭を見れば、どれが本当に形を変えたか分かる
- ・ 毛を Karma で出す — 既定の太さは、画の上で 0.1ピクセルしかなかった
- ・ 毛に色を付ける — 光を消して測ると、色は位置ぴったりに乗った
- ・ ターンテーブルで見せる — 形は物を回してもカメラを回しても同じ。変わるのは光だけ
- ・ 文字を光らせる — 明るさは頭打ち、こぼれる光と時間だけが增える
- ・ 箱 100 個を USD にすると、パックしなければメッシュ 1 つ、Point Instancer なら 6 プリム、Xforms なら 202 プリム
- ・ 箱 1 万個の USD — Point Instancer は 0.32 MB・8 ミリ秒、Native Instances は 4.6 倍、Unpack は 7.4 倍の大きさ

Karma でのレンダリング – ノイズはサンプル数で本当に減るのか

2026-09-24 | Houdini 21.0.700 | /obj/pyro_test | ノード 6 / 接続 5

何を作ったか

実験016で煙を描いたとき、ハードウェアレンダラー（OpenGL ROP）では濃淡が分らないと書いた。そこで本式のレンダラーである Karma を使う。ただ「きれいになった」で終わらせず、測れるものを測る。

Karma は1ピクセルの色を、乱数で飛ばした何本もの光線の平均で決める。平均のばらつきは本数の平方根に反比例するはずなので、サンプル数を4倍にすればノイズは半分になるという式が立つ。これを実測で確かめた。

結果、式が当たるのは狭い範囲だけだった。そして016で書いた「ハードウェアレンダラーはボリウムを描けない」も、言い過ぎだったと分かった。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
emitter	sphere	radx = 0.5 rady = 0.5 radz = 0.5 ty = 0.6 type = 1
source	pyrosource	attribute1 = 0 attributes = 1 mode = 2
rasterize	volumerasterizeattributes	attributes = density folder1 = 1 folder3 = 1 voxelsize = 0.08

ノード	タイプ	デフォルトから変更したパラメータ
solve_d0_0	pyrosolver	_folder32_1 = 1 color_diss_controlramp1interp = 3 color_diss_controlramp2interp = 3 color_diss_controlramp2pos = 1.0 color_diss_controlramp2value = 1.0 density_ramp2pos = 1.0 density_ramp2value = 1.0 dissipation = 0.0 dissipation_controlramp2pos = 1.0 dissipation_controlramp2value = 1.0 dissipation_remapcontrol = 1 disturbance_controlramp2pos = 1.0 …ほか 102 件
solve_d0_1	pyrosolver	_folder32_1 = 1 color_diss_controlramp1interp = 3 color_diss_controlramp2interp = 3 color_diss_controlramp2pos = 1.0 color_diss_controlramp2value = 1.0 density_ramp2pos = 1.0 density_ramp2value = 1.0 dissipation_controlramp2pos = 1.0 dissipation_controlramp2value = 1.0 dissipation_remapcontrol = 1 disturbance_controlramp2pos = 1.0 disturbance_controlramp2value = 1.0 …ほか 100 件
solve_d0_3	pyrosolver	_folder32_1 = 1 color_diss_controlramp1interp = 3 color_diss_controlramp2interp = 3 color_diss_controlramp2pos = 1.0 color_diss_controlramp2value = 1.0 density_ramp2pos = 1.0 density_ramp2value = 1.0 dissipation = 0.3 dissipation_controlramp2pos = 1.0 dissipation_controlramp2value = 1.0 dissipation_remapcontrol = 1 disturbance_controlramp2pos = 1.0 …ほか 101 件

サンプル数を変えたときのノイズ

ノイズの量は、基準画との差ではなく、3枚を連立させて1枚ずつ求めた。基準画にもノイズが残っている
ので、基準画に頼ると結果がその出来に左右されるため。640×360、煙とみなした画素は 21,525 個。

サンプル数	ノイズ	前段との比	式の予測	ずれ
2	18.316	—	—	—
4	11.321	0.6181	0.7071	−12.6%
8	6.516	0.5756	0.7071	−18.6%
16	4.559	0.6997	0.7071	−1.0%
32	3.413	0.7486	0.7071	+5.9%
64	2.640	0.7734	0.7071	+9.4%
128	2.345	0.8883	0.7071	+25.6%

サンプル数	ノイズ	前段との比	式の予測	ずれ
256	2.088	0.8904	0.7071	+25.9%

サンプル数を4倍にしたとき（式なら 0.5000）

8→32 の区間だけが式どおり。その下では式より速く、その上では遅く減る。

区間	実測の比	式との差
2 → 8	0.3558	式より速く減る
4 → 16	0.4028	式より速く減る
8 → 32	0.5238	ほぼ式どおり
16 → 64	0.5789	やや遅い
32 → 128	0.6870	明らかに遅い
64 → 256	0.7909	ほとんど減らない

指定したサンプル数と、実際に使われたサンプル数

2～256 の範囲で時間を直線に当てはめると「固定費 2.26秒 + 1サンプルあたり 7.44ミリ秒」になる。この直線から逆算した実効サンプル数を右端に置いた。指定を64倍にしても、実際には6倍にもなっていない。

指定	実測の時間	直線の予測	実測/予測	実効サンプル
64	3.00 秒	2.74 秒	1.10	99
128	3.09 秒	3.21 秒	0.96	112
256	3.43 秒	4.16 秒	0.82	157
512	4.23 秒	6.07 秒	0.70	264
1024	5.52 秒	9.88 秒	0.56	439
2048	5.94 秒	17.50 秒	0.34	495
4096	6.48 秒	32.73 秒	0.20	567

頭打ちの犯人 — varianceaa_thresh

samplesperpixel を 4096 に固定したまま、打ち切りのしきい値だけを下げた。しきい値を50分の1にすると、実際の仕事量は21.9倍になった。頭打ちはハードウェアの限界ではなく、この設定によるもの。

varianceaa_thresh	時間	実効サンプル	既定との比
0.01（既定）	5.39 秒	420	1.00
0.005	6.29 秒	542	1.29
0.001	8.01 秒	773	1.84
0.0002	70.64 秒	9,191	21.87

同じ煙を2つのレンダラーで — 同じカメラ・同じ露出

「煙の画素」は真っ白でない画素の数。「最も暗い」は、その画素がどこまで濃く描かれたか。

レンダラー	時間	煙の画素	階調	平均の明るさ	ばらつき	最も暗い
OpenGL ROP	0.71 秒	29,242	235 段	101.0	86.16	15
Karma 256サンプル	3.64 秒	21,545	200 段	109.0	52.83	50

煙の中心を横切る線の明るさ

左から右へ等間隔に見たときの値。255 が背景。数字が小さいほど濃い。

レンダラー	縁の手前	縁	中へ	中心	中心	中へ	縁	縁の外
OpenGL ROP	254	224	121	58	42	50	112	219
Karma	255	255	184	109	104	104	189	255

わかったこと

- ・式が当たったのは 8〜32 の区間だけだった。サンプル数を4倍にしたときの比は、8→32 で 0.5238（式は 0.5000）とよく一致する。ところが 64→256 では 0.7909 で、ほとんど減らない。

- ・減らなくなる理由を突き止めた。samplesperpixel

は「ここまで使ってよい」という上限であって、「必ずこれだけ使う」という指定ではない。指定を 64 から 4096 へ64倍にしても、時間から逆算した実効サンプル数は 99 から 567 へ5.7倍にしかならなかった。

- ・犯人は varianceaa_thresh（既定

0.01）だった。これは「このくらい収束したら、そのピクセルはもう打ち切る」というしきい値。4096 を指定したまましきい値を 0.0002 まで下げると、実効サンプル数は 420 から 9,191 へ21.9倍になり、時間も 5.39秒から70.64秒に伸びた。上限ではなく打ち切りの設定が効いていた。

- ・実用的な結論。この場面では 32 サンプルあたりで費用対効果が折り返す。それ以上は、数字を上げるだけでは絵は良くなる。本当に詰めたいときは varianceaa_thresh も一緒に下げる。ただし時間は一気に13倍になる。

- ・時間の内訳も分かった。640×360 で固定費が 2.26 秒、1サンプルあたり 7.44 ミリ秒。2サンプルでは時間のほぼ全部が固定費で、256サンプルでも半分強が固定費だった。軽い絵を何枚も撮るときは、サンプル数より枚数のほうが効く。

- ・実験016に書いた「ハードウェアレンダラーはボリュームを描けない」は言い過ぎだった。同じカメラ・同じ露出で撮り直すと、OpenGL ROP も煙の輪郭と、厚みによる濃淡（中心ほど濃い）を描けている。中心を横切る線を見ると、縁の 254 から中心の 42 までなだらかに変化している。

- ・2つのレンダラーの本当の違いは、光が中を通るかどうかだった。OpenGL は濃さの分だけ暗くするだけで、どちらから光が当たっているかが絵に出ない。Karma は光源側が明るく、反対側が暗い。Karma のほうが最も暗い画素が 50 と薄いのも、光が内部を通して抜けてくるため。

- ・速さの差は大きい。OpenGL が 0.71 秒に対し、Karma は 256サンプルで 3.64 秒。5倍。確認のための描画は今後も OpenGL でよく、Karma は見せる絵に使う。

- ・測り方で3回つまずいた。(1) 出力画像には透明度が付いていて、透明部分の色は黒だった。白い背景に見えるのは表示側が白で合成しているため。これに気づかず「白から離れた画素＝煙」というマスクを作り、画面の88%を拾って、ほとんど何も測っていなかった。

- ・ (2) 煙の芯が真っ白 (255) に張り付いていた。張り付いた画素は元の値が分からないので、ノイズを測っても意味がない。ライトを下げ、中間調の画素だけを対象にした。

- ・ (3) OpenGL と Karma でカメラの寄り方が違っていた (margin 1.12 対 0.5) 。煙の写る画素数が違って当たり前で、最初の比較は無効だった。実験003以降ずっと守ってきた「全変種を同じカメラで撮る」を、自分で破っていた。

- ・ Apprentice でも Karma

は問題なく動く。出力には右下にウォーターマークが入るので、測る範囲からは外している。

次にやること

- ・ 構造のある煙を作って撮り直す。いまの煙はほぼ一様な球で、レンダラーの差を見せる題材として弱い

- ・ 018で決めた Karma の設定 (denoiser off、varianceaa_thresh、露出) を hou_tools に関数としてまとめる

- ・ 019 として FLIP に再挑戦する。付属ヘルプで flipsolver の入力の作法を読む

浸食のレイヤーで地形を塗り分ける ー 色が地形の形と対応しているか

2026-09-24 | Houdini 21.0.700 | /obj/terrain | ノード 5 / 接続 5

何を作ったか

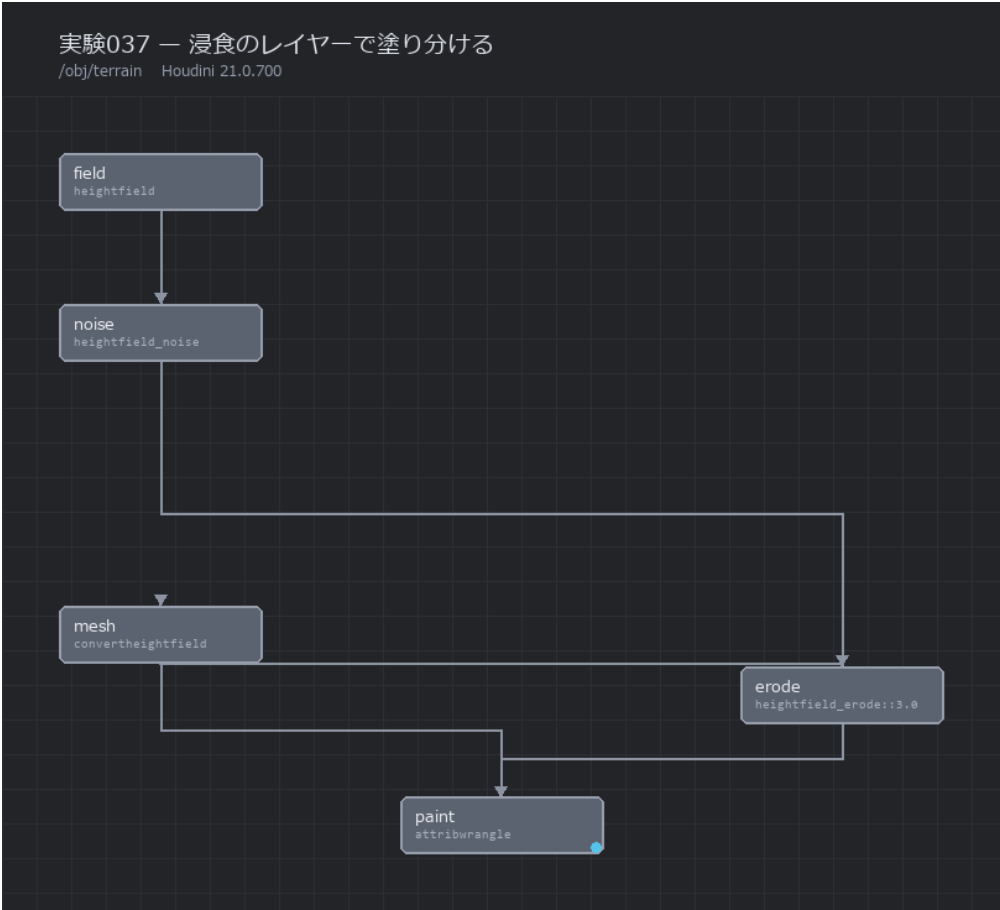
実験034で、浸食が過程をレイヤーとして残すことが分かった。035では sediment を散布の重みに、036では flowdir を向きに使った。今回は色にする。

見た目を作るだけなら適当に塗っても絵にはなる。そうではなく、塗り分けが地形の形と対応しているかを数字で確かめた。

対応していた。岩肌に塗った場所の勾配の平均は 0.44733、土に塗った場所は 0.07076 で、岩肌は土の 6.32倍 急である。水筋はその中間の 0.21210 で、これも筋が通る（水は斜面を下って谷に集まる）。

しきい値は勘で決めず、レイヤーの値の分布を見てから決めた。sediment は半分以上のマスがゼロで、下から75%でようやく 0.8157。この形を見てから 0.30 を境にした。

ノードの接続



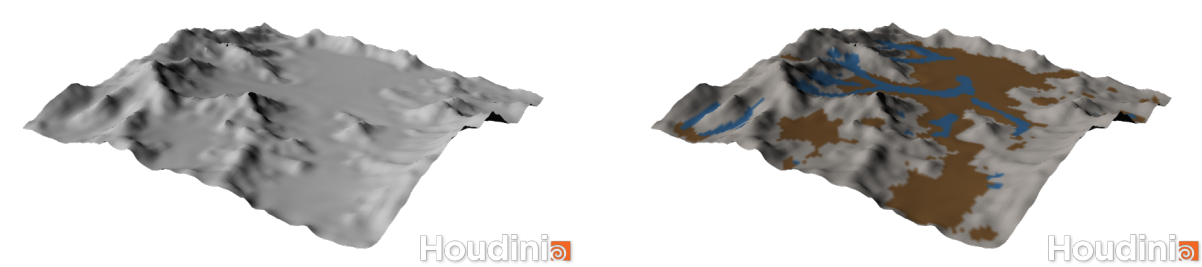
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
field	heightfield	sizeX = 200.0 sizeY = 200.0
noise	heightfield_noise	amp = 120.0 elementsSize = 80.0

ノード	タイプ	デフォルトから変更したパラメータ
erode	heightfield_erode::3.0	folder0 = 1 folder1 = 1 iterations = 4 seed = 0.2140711545944214
mesh	convertheightfield	(変更なし)
paint	attribwrangle	snippet = f@sediment2 = volumesample(1, "sediment", @P); f@flow2 = volumesample(1, "flow", @P); f@debris2 = volumesample(1, "de...

塗り分けのあるなし

同じ地形。違いは @Cd に何を入れるかだけ。カメラは2枚とも同じ。



塗り分け無し。形は見えるが、どこが土でどこが岩か分からない。浸食のレイヤーで塗り分け。灰 = 岩肌、茶 = 土、青 = 水筋。

色	意味	条件
灰 (0.62, 0.60, 0.58)	岩肌。土が流れ去って何も溜まっていない	どちらにも当てはまらない
茶 (0.45, 0.32, 0.18)	土。堆積が積もった	sediment ≥ 0.30
青 (0.20, 0.38, 0.55)	水筋。水が集まって流れた	flow ≥ 1.60 (こちらが優先)

1. しきい値を決めるために、値の分布を見る

100×100 のマス目 10,000個ぶんの値を、下から何%の位置かで見た。

レイヤー	最小	下から10%	25%	50%	75%	90%	最大
sediment	0.0000	0.0000	0.0000	0.0000	0.8157	2.9817	11.0788
flow	0.0000	0.0969	0.3697	0.6956	1.1336	1.5871	4.1504
debris	0.0000	0.0000	0.0000	0.0000	0.0014	0.0854	2.5569

2. 分けた3つは、勾配で見て別のものか

塗り分けたあとで、それぞれの勾配・堆積・流れの平均を出した。勾配は高さから直接作っている。

種類	点の数	割合	勾配の平均	勾配の中央値	堆積の平均	流れの平均
岩肌	6,370	63.7%	0.44733	0.40695	0.01660	0.59386
土	2,653	26.5%	0.07076	0.05448	2.39543	0.83297
水筋	977	9.8%	0.21210	0.13315	1.93003	2.06320

わかったこと

- ・塗り分けは地形の形と対応していた。岩肌の勾配の平均が 0.44733、土が 0.07076。岩肌は土の 6.32倍 急である。中央値で見ても 0.40695 対 0.05448 で 7.5倍。色は模様ではなく、形の情報を持っている。
- ・水筋は中間に入った。勾配の平均 0.21210 で、岩肌より緩く土より急。水は斜面を下って谷に集まるので、その通り道は「急でも緩でもない途中」になる。狙って作った並びではないが、筋が通っている。
- ・しきい値は勘で決めない。まずレイヤーの値の分布を出した。sediment は半分以上のマスがゼロで、下から75%の位置でようやく 0.8157。この形を見てから 0.30 を境にした。分布を見ずに「0.5くらい」と決めていたら、土がほとんど出ないところだった。
- ・debris は使えなかった。下から90%の位置でも 0.0854 しかなく、ほとんどのマスが実質ゼロ。レイヤーがあることと、塗り分けに使えることは別の話。
- ・flow は逆に、どこにでも値がある。下から10%でも 0.0969。実験035で「100%のマスに値がある」と測ったとおりで、ゼロとの比較では分けられない。上位の値（1.60 以上、全体の約1割）で切って初めて筋になる。
- ・優先順位を決める必要があった。水筋のところは堆積も多い（堆積の平均 1.93）ので、条件が重なる。水筋を先に判定して、残りを土と岩肌に分けた。順番を逆にすると水筋が土に飲まれて消える。
- ・使われた色はちょうど3種で、割合は 63.7% / 26.5% / 9.8%。塗り残しも、意図しない中間色も無い。
- ・勾配はnormal SOP の N からではなく、高さのボリュームから隣との差で作った。normal SOP は N を点ではなくバーテックスに書くため（実験030・036）、点のラングルからは読めない。

次にやること

- ・岩肌と土で、表面の細かさ（ノイズ）を変える
- ・この塗り分けを Karma で見せる絵にする
- ・038 以降、まだ触っていない分野に進む

塗り分けた地形を Karma で出す — 見立ては外れ、白飛びが本命だった

2026-09-24 | Houdini 21.0.700 | /obj/terrain | ノード 6 / 接続 6

何を作ったか

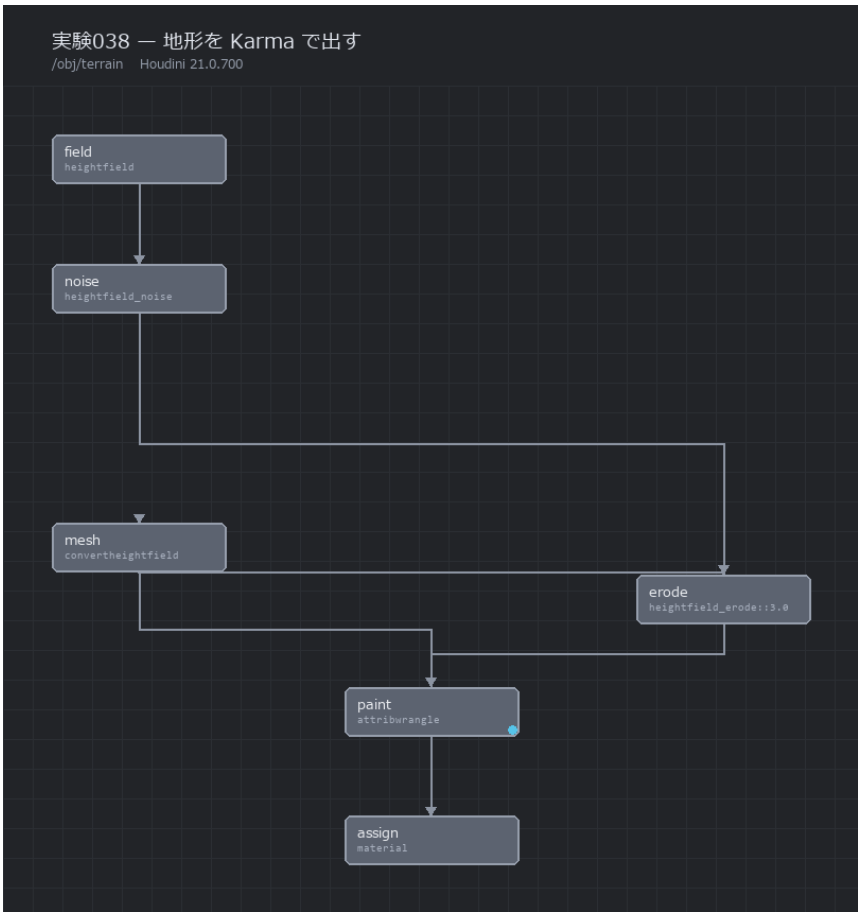
実験037で地形を3色に塗り分けたが、あれは確認用の速い描画（OpenGL）で、点に付けた @Cd がそのまま画になっていた。見せる絵にするときは Karma を使う。

見立ては「材質を割り当てないと点の色は出ない」だった。外れた。材質なしでも色は出る。principled shader の「点の色を使う」も既定でオンで、割り当てても色の出方は変わらない。

代わりに別の問題が出た。白飛びである。材質を割り当てないと、地形の 43.4% の画素が白く飛ぶ。割り当てると 3.1% に落ちる。14倍の差で、これは色ではなく明るさの話だった。

さらに、白飛びの原因はライトではなかった。材質を割り当てたあとはキーライトを 2.4 から 0.3 へ8分の1にしても、白飛びは 3.0% から 2.9% にしか変わらない。全体の明るさは 167.66 から 110.22 へ下がるのに、飛んでいるところは飛んだままである。

ノードの接続



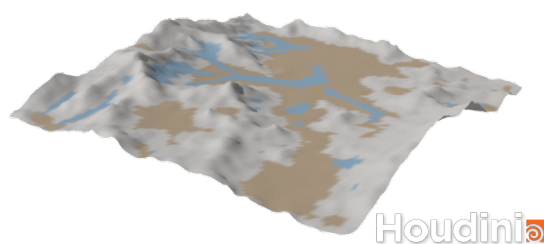
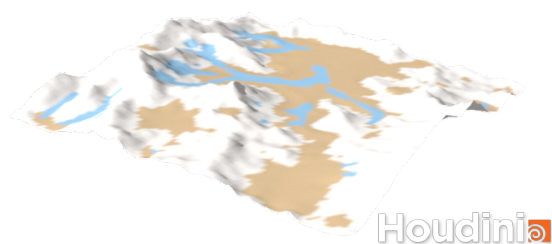
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
field	heightfield	size_x = 200.0 size_y = 200.0

ノード	タイプ	デフォルトから変更したパラメータ
noise	heightfield_noise	amp = 120.0 elementsize = 80.0
erode	heightfield_erode::3.0	folder0 = 1 folder1 = 1 iterations = 4 seed = 0.543621301651001
mesh	convertheightfield	(変更なし)
paint	attribwrangle	snippet = float sed = volumesample(1, "sediment", @P); float flw = volumesample(1, "flow", @P); int water = (flw >= 1.6) ? 1 : ...
assign	material	shop_materialpath1 = /mat/terrain_mat

材質のあるなし（ライトは同じ）

実験024で煙に合わせたライト設定のまま。カメラも解像度も同じ。違いは材質を割り当てるかどうかだけ。



材質なし。色は出るが、岩肌が真っ白に飛ぶ（43.4%）。

材質あり。同じライト。白飛びは 3.1% まで落ちる。

	時間	白飛び	茶	青	灰
材質なし	4.13秒	43.4%	27.4%	7.2%	65.5%
材質あり	3.55秒	3.1%	24.7%	6.4%	68.9%

1. ライトを弱くすると、白飛びは減るか

材質を割り当てた状態で、キーライトだけを8分の1まで落とした。ドームライトは 1.0 で固定。

キーライト	時間	白飛び	明るさ	ばらつき	茶	青
2.4	3.40秒	3.0%	167.66	26.30	23.8%	6.2%
1.2	3.35秒	3.0%	138.95	25.89	22.9%	5.5%
0.6	3.28秒	3.0%	120.93	26.91	21.0%	5.0%
0.3	3.20秒	2.9%	110.22	28.04	19.7%	4.3%

2. OpenGL と Karma

同じカメラ・同じ解像度（720×460）。Karma はキーライト 0.3、サンプル 24。

	時間	白飛び	明るさ	ばらつき	茶	青
OpenGL	1.29秒	2.9%	103.50	44.51	27.4%	6.8%

	時間	白飛び	明るさ	ばらつき	茶	青
Karma	3.20秒	2.9%	110.22	28.04	19.7%	4.3%
比	2.5倍	同じ	1.06倍	0.63倍	—	—

わかったこと

- ・見立ては外れた。点の色は材質なしでも出る。「材質が無いと Karma は色を出しようがない」と考えていたが、既定の材質がちゃんと @Cd を読む。principled shader の「点の色を使う」も既定でオンなので、割り当てても色の出方は変わらない。
- ・代わりに白飛びが出た。材質を割り当てないと、地形の 43.4% の画素が白く飛ぶ。割り当てると 3.1%。14倍の差で、これは色ではなく明るさの話だった。
- ・白飛びの原因はライトではなかった。材質を割り当てたあととは、キーライトを 2.4 から 0.3 へ8分の1にしても、白飛びは 3.0% から 2.9% にしか変わらない。全体の明るさは 167.66 → 110.22 と下がるのに、飛んでいるところは飛んだまま。ライトを絞っても直らない飛び方がある。
- ・測り方で一度失敗した。最初は「明るい画素は背景」として地形と背景を分けたが、白飛びした地形まで背景として捨ててしまい、白飛びが 0.0% と出た。書き出したPNGは透明度を持っているので、そちらで分け直したら 43.4% になった。測りたいものが「真っ白」なのに、真っ白を捨てる仕分けをしていた。
- ・Karma は OpenGL の 2.5倍の時間。720×460・サンプル24 で 3.20秒 対 1.29秒。実験018・024で煙を撮ったときは5倍前後だったので、面だけの地形はボリュームより差が小さい。
- ・ばらつき（明暗の差）は Karma のほうが小さい。44.51 対 28.04 で 0.63倍。ドームライトが影を埋めるためで、「本番のレンダのほうがコントラストが強い」とは限らない。
- ・ライトは題材ごとに決め直す。実験024で煙に合わせた設定（キー 2.4）をそのまま地形に使うと飛ぶ。同じ設定を使い回せるのは、同じ種類のものを撮るときだけ。

次にやること

- ・ドームライトの強さも振って、影の出方を測る
- ・地形に空（環境マップ）を入れて、水面に映り込ませる
- ・039 以降、まだ触っていない分野に進む

Copernicus でテクスチャを作る ー 細かさは解像度をどう食うのか

2026-09-24 | Houdini 21.0.700 | /obj/cops | ノード 2 / 接続 1

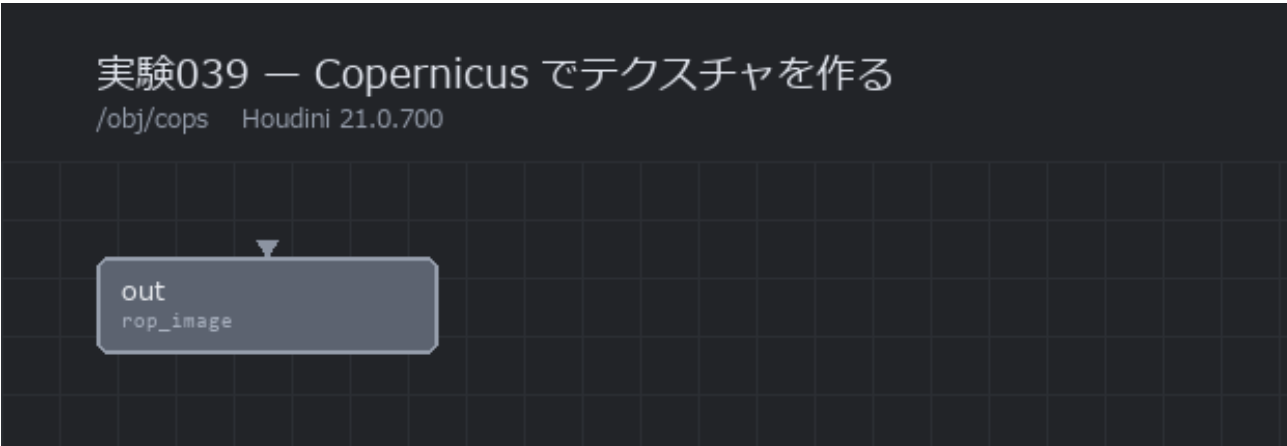
何を作ったか

ここまではすべて形（ジオメトリ）の話だった。Houdini には画そのものを作る仕組みもある。Copernicus（COP）で、専用のノードが262種ある。地形に貼る模様や汚し、マスクを、外で作って読み込むのではなく Houdini の中で手続きとして作れる。

解像度はノイズのノードではなく、ネットワーク（copnet）の側で決まる。しかも「解像度を指定する」のスイッチは既定でオフで、そのときは欄に書いてある 1024×1024 がそのまま使われる。オフなのに値は効いている。

細かさの指標は、実験007と同じ問いを投げた。画素どうしの差をそのまま見ると、解像度を2倍にするたびに約半分になる（6.4940 → 1.0266）。ただしこれは画素が細かくなったぶん隣が近づいただけで、解像度を掛けて「画全体を1とした長さあたりの差」に直すと 831.2 → 945.0 → 1038.4 → 1051.3 と増えて頭打ちになる。増えなくなったところが、その模様に足りている解像度である。実験007で地形について言ったことが、画でもそのまま成り立つ。

ノードの接続



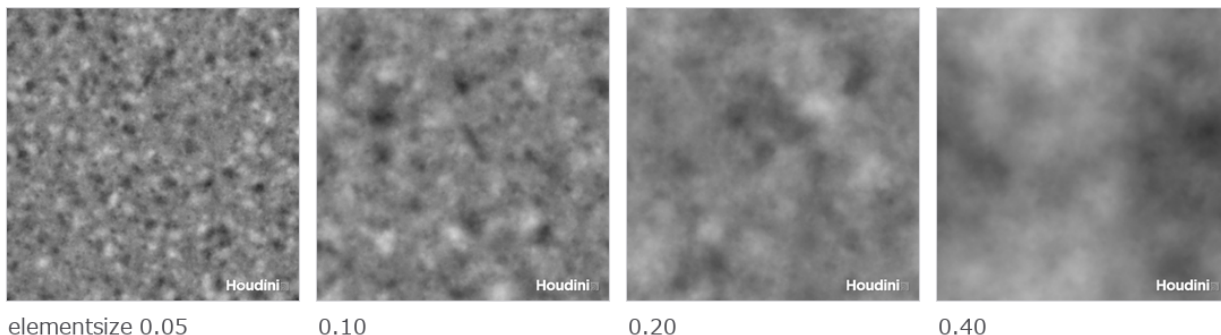
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
noise	fractalnoise	folder0 = 1 folder1 = 1 folder2 = 1
out	rop_image	aov1 = C outputaovs = 1

パラメータを変えたときの見え方

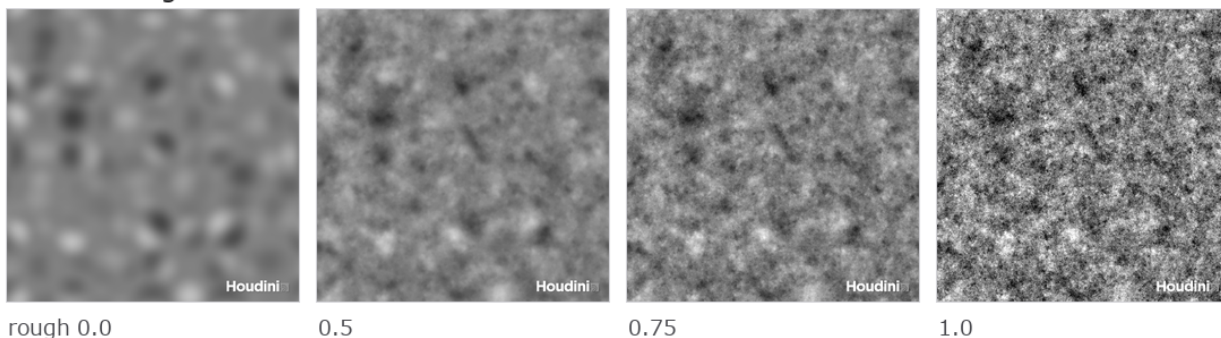
どちらも 1024×1024。上段はうねりの大きさ、下段は細かさを変えている。

うねりの大きさ (elementsize) を変える



elementsizes 0.05 / 0.10 / 0.20 / 0.40。数字が大きいくほど、うねりが大きくなる。

細かさ (rough) を変える



rough 0.0 / 0.5 / 0.75 / 1.0。1.0 では砂嵐に近くなる。

項目	値
使ったノード	copnet → fractalnoise → rop_image
Copernicus のノード	262種
解像度を決める場所	copnet の res1 / res2 (ノイズ側には無い)
書き出した階調	206～215 (8ビットPNG の上限は256)

1. 解像度はどこで決まるか

copnet の「解像度を指定する (setres)」と res1/res2 を変えて、書き出した画の大きさを見た。

設定	書き出した大きさ	階調の数
setres オフ (既定)	1024×1024	210
setres オン / 256	256×256	206
setres オン / 512	512×512	215
setres オン / 1024	1024×1024	210

2. うねりの大きさ (elementsize)

1024×1024 で elementsizes だけを変えた。細かさ = 隣り合う画素の差の平均 (0～255の目盛)。

elementsizes	明るさ	ばらつき	細かさ
0.05	128.00	20.69	1.7913
0.10	129.22	19.79	1.0266

elementsize	明るさ	ばらつき	細かさ
0.20	128.43	16.94	0.5752
0.40	127.50	24.88	0.3657

3. 細かさ (rough)

1024×1024 で rough だけを変えた。

rough	明るさ	ばらつき	細かさ
0.0	129.28	16.92	0.4020
0.25	129.23	17.58	0.4841
0.50	129.22	19.79	1.0266
0.75	129.25	25.72	5.6503
1.0	129.29	49.17	32.9725

4. 細さは解像度によらない量になるか（実験007と同じ問い）

同じノイズ (elementsize 0.10) を解像度だけ変えて書き出した。「細かさ×解像度」は、画全体を1とした長さあたりの差にあたる。

解像度	細かさ	前の行との比	細かさ×解像度	前の行との比	ばらつき
128	6.4940	—	831.2	—	22.77
256	3.6914	0.568	945.0	1.137	22.76
512	2.0280	0.549	1038.4	1.099	22.76
1024	1.0266	0.506	1051.3	1.012	19.79

わかったこと

- ・ 解像度はノイズではなくネットワークの側で決まる。fractalnoise に解像度の欄は無い。copnet の res1 / res2 が決めている。形 (SOP) の感覚で「そのノードに設定がある」と探すと見つからない。
- ・ 「解像度を指定する」は既定でオフなのに、値は効いている。オフのままでも 1024×1024 で出てくる。欄に書いてある既定値がそのまま使われるため、オフ=無効ではない。
- ・ 画素どうしの差は、解像度を2倍にするたび約半分になる。6.4940 → 3.6914 → 2.0280 → 1.0266 で、比は 0.568 / 0.549 / 0.506。これは模様が変わったのではなく、画素が細かくなって隣が近づいただけである。
- ・ 解像度を掛けて直すと、実験007と同じ形になった。831.2 → 945.0 → 1038.4 → 1051.3 で、比は 1.137 / 1.099 / 1.012。増えて頭打ちになる。頭打ちになったところが、その模様に足りている解像度で、elementsize 0.10 なら 1024 で足りている。
- ・ ばらつきは解像度によらずほぼ一定だった。22.77 / 22.76 / 22.76 / 19.79。「どれだけ濃淡があるか」は解像度で変わらないが、「どれだけ細かいか」は変わる。実験007で「高さの平均は早く収束するのに勾配は伸び続ける」と測ったのと同じ関係になる。
- ・ elementsize は素直に効く。0.05 から 0.40 へ8倍にすると、細さは 1.7913 から 0.3657 へ約5分の1。大きくするほどうねりが大きく、なだらかになる。

- ・ rough の効き方は急である。0.0 で 0.4020、1.0 で 32.9725。82倍になる。0.5 までは穏やかで、0.75 から一気に立ち上がる。

- ・ ただし rough は明るさのばらつきも変えてしまう。16.92 → 49.17 で約2.9倍。実験004で mountain の rough を測ったときは「振幅を変えずに形の性質だけを変える」だったが、COP のノイズでは同じ名前でも振る舞いが違う。名前が同じでも、別のノードなら測り直す。

- ・ 書き出した画の階調は 206~215 だった。8ビットPNG の上限は256なので、ほぼ使い切っている。ただし値は 0~255 に丸められるので、マスクとして精度が要るなら EXR など小数のまま持てる形式にする。

次にやること

- ・ 作ったテクスチャを地形の材質に貼って、Karma で確かめる
- ・ COP のノイズを HeightField の高さに流し込む
- ・ 040 以降、まだ触っていない分野に進む

作ったテクスチャを地形に貼る — 貼れたかどうかを画の数字で判定する

2026-09-24 | Houdini 21.0.700 | /obj/terrain | ノード 5 / 接続 4

何を作ったか

実験039で Copernicus のノイズを画像として書き出した。実験038で地形を Karma で出した。今回はその2つをつなぐ。

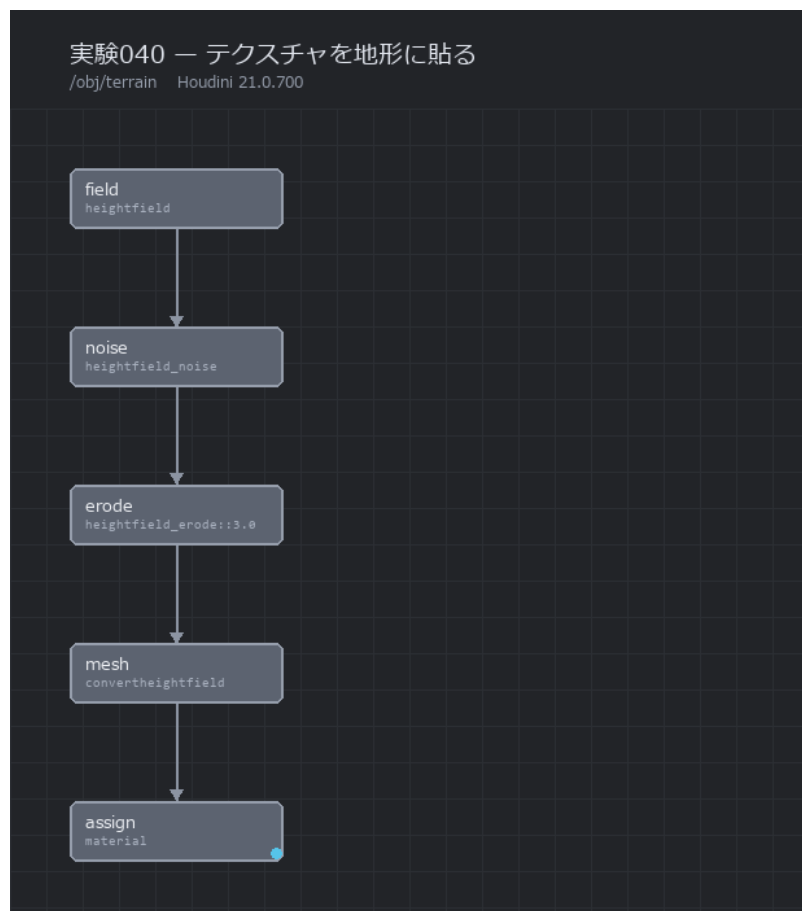
「貼ったつもりで貼れていない」はよくある。そこで貼れたかどうかを画の数字で判定した。テクスチャを貼れば面の中に細かい濃淡が出るはずなので、貼る前と貼ったあとで画の細かさが増えるはずである。増えなければ貼れていない。

増えた。細かさは 2.2143 から 3.0095 へ 1.36倍、濃淡のばらつきは 14.83 から 27.54 へ 1.86倍。貼れている。

ただしテクスチャの解像度を 256 から 1024 へ4倍にしても、画に出る細かさは 2.9028 から 3.0095 で 1.04倍にしかない。この見え方の大きさなら 256 でほぼ足りている。実験039で「頭打ちになったところが足りている解像度」と測ったことが、貼る側でも成り立つ。

もう1つ、Apprentice では COP から書き出した画像にウォーターマークが入る。それを貼ると地形の表面にも写る。

ノードの接続



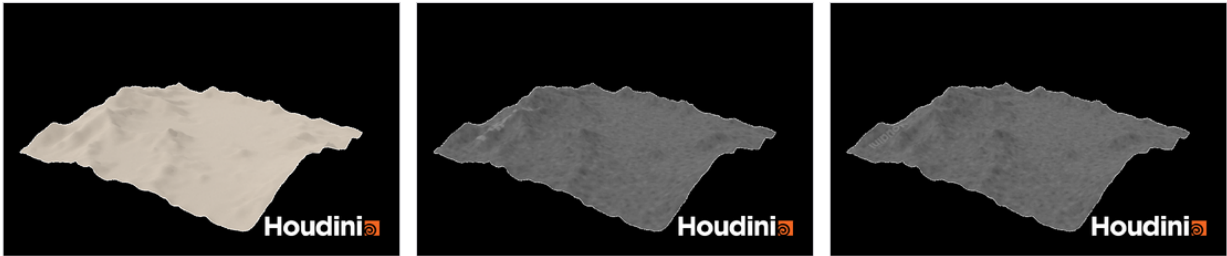
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
field	heightfield	sizeX = 200.0 sizeY = 200.0
noise	heightfield_noise	amp = 120.0 elementSize = 80.0
erode	heightfield_erode::3.0	folder0 = 1 folder1 = 1 iterations = 4 seed = 0.32275283336639404
mesh	convertHeightfield	(変更なし)
assign	material	shop_materialPath1 = /mat/terrain_mat

貼る前と貼ったあと

同じ地形・同じライト・同じカメラ。違いは貼る画だけ。

同じ地形・同じライト・同じカメラ。違いは貼る画だけ



テクスチャなし 細かさ 2.2143

256×256 2.9028

1024×1024 3.0095

左から、テクスチャなし / 256×256 / 1024×1024。

テクスチャ	時間	明るさ	ばらつき	細かさ
なし（一色）	5.35秒	191.73	14.83	2.2143
256×256	3.44秒	110.31	27.56	2.9028
1024×1024	3.47秒	110.26	27.54	3.0095

地形は UV を持っているか

テクスチャを貼るには UV が要る。convertHeightfield の出力を調べた。

調べたこと	結果
点アトリビュート	P, debris, flow, flowdir_x, flowdir_y, height, mask, sediment, uv
uv はあるか	ある（自分で作らなくてよい）
テクスチャを作ったノード	copnet → fractalnoise → rop_image（実験039と同じ）
貼り付け先	principledshader の basecolor_useTexture / basecolor_texture

わかったこと

- ・貼れたことを、見た目ではなく数字で確かめた。細かさは 2.2143 → 3.0095 で 1.36倍、ばらつきは 14.83 → 27.54 で 1.86倍。「貼ったつもり」で終わらせないために、増えるはずの量を先に決めておく。

- ・テクスチャの解像度を4倍にしても、画はほとんど変わらなかった。256 で 2.9028、1024 で 3.0095。1.04倍にしかない。この見え方の大きさなら 256 でほぼ足りている。

- ・実験039の結論が、貼る側でも成り立つ。039では「細かさ×解像度が頭打ちになったところが、その模様に足りている解像度」と測った。今回は画面に出る大きさが上限を決めている。地形が画面の一部しか占めていないので、それ以上細かいテクスチャは画に出ない。

- ・UV は自分で作らなくてよい。convertheightfield の出力に uv が最初から付いている。実験036で「レイヤーが点アトリビュートとして持ち込まれる」と測ったときと同じ一覧に入っている。

- ・Apprentice では、COP から書き出した画像にウォーターマークが入る。それを貼ると地形の表面に文字が写る。レンダの隅に出るウォーターマークと違い、テクスチャに焼き込まれているので、地形のどこにでも現れる。学習用に作る画としては問題ないが、本番で使う画をこの経路で作ることはできない。

- ・一色のときのほうが遅かった。5.35秒 対

3.44秒。テクスチャを貼ったほうが速い。理由は測っていないが、一色のほうが明るく（191.73 対 110.26）、明るい面ほど光が何度も跳ね返って計算が増えるためと考えられる。ここは確かめていないので断定しない。

- ・貼ると暗くなる。明るさ 191.73 → 110.26。作ったノイズの平均が中間の灰色で、一色に指定した (0.55, 0.48, 0.40)

より暗いため。テクスチャに差し替えると明るさが変わるので、ライトは貼ったあとに決める。

次にやること

- ・ノイズを高さ (displacement) として使い、面そのものを凸凹させる
- ・岩肌と土で別のテクスチャを使い分ける (037の塗り分けをマスクにする)
- ・041 以降、まだ触っていない分野に進む

凸凹の付け方3通り —

輪郭を見れば、どれが本当に形を変えたか分かる

2026-09-24 | Houdini 21.0.700 | /obj/terrain | ノード 6 / 接続 6

何を作ったか

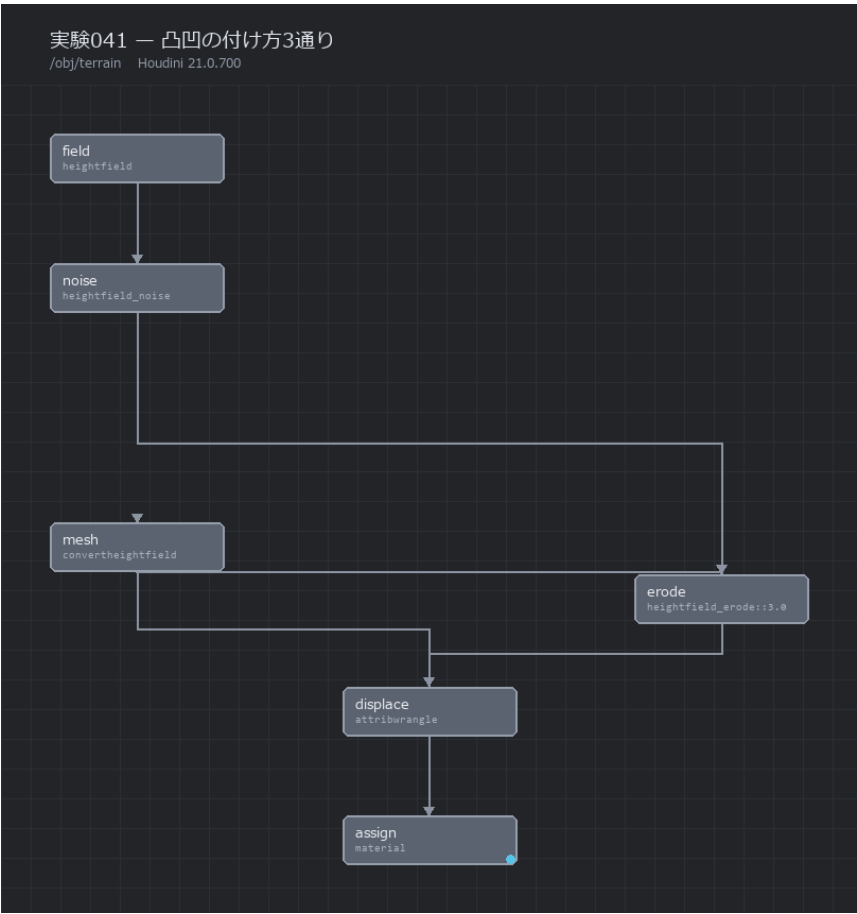
面を凸凹させる方法は3つある。ジオメトリを動かす（点そのものを動かす）、displacement（レンダるときだけ面を動かす）、bump（面は動かさず光の当たり方だけ凸凹に見せる）。

見た目はどれも似る。決定的な違いは輪郭（シルエット）に出るはずである。本当に面が動いていればフチがぎざぎざになり、光のごまかしならフチは滑らかなままになる。そこで輪郭の長さを測った。書き出した画の透明度から、地形と背景の境目になっている画素を数える。

狙いどおりに分かれた。bump は画の細かさを一番大きく増やす（2.2945 → 4.0524 で 1.77倍）のに、輪郭は 0.999倍でまったく変わらない。displacement は点を1ミリも動かさないのに輪郭を 1.020倍にする。ジオメトリを動かした場合だけ、点の位置そのものが変わった（最大 2.22）。

「見た目が凸凹しているか」と「形が凸凹しているか」は別のことで、輪郭を測れば見分けられる。

ノードの接続



スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

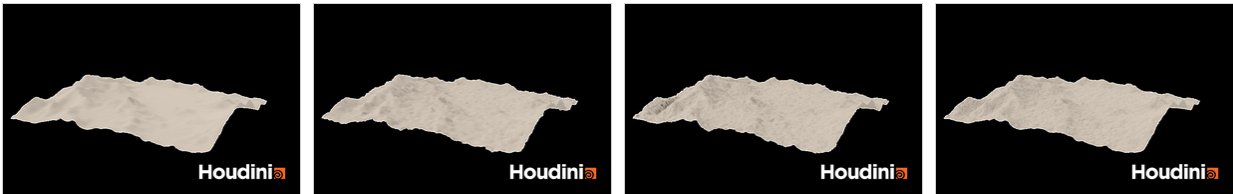
ノード	タイプ	デフォルトから変更したパラメータ
field	heightfield	size _x = 200.0 size _y = 200.0

ノード	タイプ	デフォルトから変更したパラメータ
noise	heightfield_noise	amp = 120.0 elementsiz = 80.0
erode	heightfield_erode::3.0	folder0 = 1 folder1 = 1 iterations = 4 seed = 0.8801623582839966
mesh	convertheightfield	(変更なし)
displace	attribwrangle	snippet = float e = 2.0; float h0 = volumesample(1, "height", @P); float hx = volumesample(1, "height", @P + set(e, 0, 0)); flo...
assign	material	shop_materialpath1 = /mat/terrain_mat

4通りを同じカメラで撮る

同じ地形・同じライト・同じカメラ。displacement と bump には、まったく同じ画（Copernicus で作ったノイズ）を渡している。

同じ地形・同じライト・同じカメラ。凸凹の付け方だけが違う



何もしない / ジオメトリを動かす / displacement / bump

左から、何もしない / ジオメトリを動かす / displacement / bump。

やり方	点が動いた量	輪郭の長さ	何もしないとの比	画の細かさ	時間
何もしない	0.000000	3,064	1.000倍	2.2945	4.32秒
ジオメトリを動かす	2.219831	3,118	1.018倍	2.7508	5.02秒
displacement	0.000000	3,126	1.020倍	3.5173	6.32秒
bump	0.000000	3,062	0.999倍	4.0524	8.32秒

わかったこと

- ・ bump は輪郭をまったく変えなかった。3,062 対 3,064 で 0.999倍。それでいて画の細かさは一番大きい（4.0524、何もしない場合の1.77倍）。凸凹して見えるのに、形は1ミリも変わっていない。
- ・ displacement は点を動かさないのに輪郭を変えた。点の移動量は 0.000000 なのに、輪郭は 1.020倍。レンダのときだけ面が動いているので、データを見ても分からない。
- ・ ジオメトリを動かした場合だけ、データが変わる。点が最大 2.22 動き、高さの幅も 28.519 から 29.562 に広がった。あとの工程（散布や当たり判定）に効くのはこれだけ。
- ・ 使い分けの目安。フチに出したい凸凹はジオメトリか displacement、面の中だけでよい細かさは bump。遠景の岩肌なら bump で足り、シルエットに出したい岩の割れ目なら displacement が要る。
- ・ bump が一番遅かった。8.32秒で、何もしない場合の 1.9倍。displacement（6.32秒）より遅い。「面を動かさないから軽い」とは限らない。理由は測っていないので断定しない。

- ・最初、bump がまったく効かなかった。有効化のスイッチだけを入れてテクスチャのパスを指定していなかったため、輪郭も細かさも「何もしない」と完全に同じ数字（3056 と 2.2916）になった。完全に同じ数字が出たら、効いていないことを疑う。

- ・輪郭の測り方。書き出した画の透明度で地形と背景を分け、隣り合う画素で「地形かどうか」が食い違う場所の数を数えた。フチがぎざぎざになるほど、この数が増える。実験038で「明るさで分けると白飛びを取りこぼす」と学んだので、透明度で分けている。

- ・差は 2% 前後と小さい。目で見比べても分からない大きさだが、数えれば順番がはっきり出る。bump だけが 1.000倍から動かず、他の2つは 1.018 / 1.020 と揃って上がる。

次にやること

- ・ displacement の量を振って、輪郭の変化が量に比例するか確かめる
- ・ 地形を寄りで撮り、bump と displacement の差が見える距離を探す
- ・ 042 以降、まだ触っていない分野に進む

毛を Karma で出す — 既定の太さは、画の上で 0.1ピクセルしかなかった

2026-09-24 | Houdini 21.0.700 | /obj/groom | ノード 7 / 接続 9

何を作ったか

hairgen は毛1本ごとに width という太さを書き出す (thickness、既定 0.001) 。その 0.001 が画の上で何ピクセルになるのかを、先に式を立ててから測った。

見かけの太さ(px) = thickness × (1世界単位あたりのピクセル数)。ただし width が「直径」なら答えはこれ、「半径」なら2倍になる。どちらかは書いていないので測るしかない。

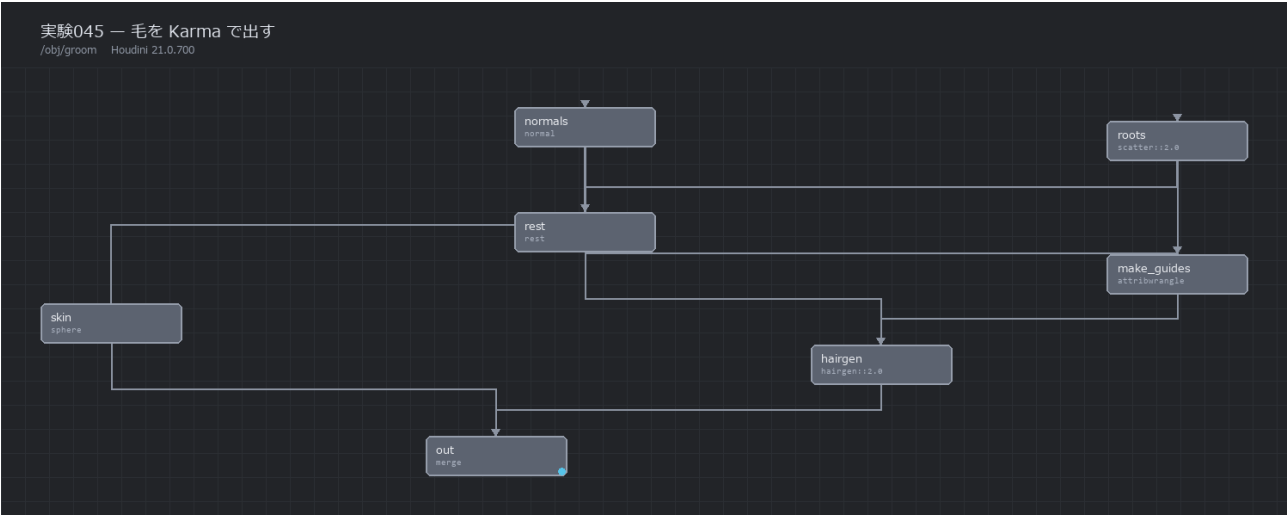
毛玉のままでは毛が重なって面積から太さを割り出せないので、毛を1本だけ置いて真横から撮った。PNG の透明度をそのまま足すと、輪郭のぼけも 0.01ピクセルまで拾える。4通りの太さで、実測÷予測は 0.9969 / 0.9969 / 0.9969 / 1.0019。width は直径だった。

ただし最初の測定は 3~4% ずれていた。Apprentice のレンダには右下に Houdini のロゴが焼き込まれ、そこにも透明度がある。列ごとに透明度を足すやり方だと、ロゴの列だけ値が水増しされていた。ロゴの箱 (行521~566、列343~566、合計 4163.3) を外したところ、4通りとも 0.3% 以内に収まった。

そのうえで既定値に戻すと、球1個を画面いっぱい撮った構図で毛の太さは 0.103ピクセル。1ピクセルの1割しかない。だから既定のままレンダするとザラついた点の集まりにしか見えない。0.008 まで上げると毛の流れが見える絵になる。

レンダ時間は本数に比例しない。毛を16倍 (3,104本 → 49,977本) にしても時間は 4.2倍 (3.1秒 → 13.1秒) 。OpenGL は本数によらず 0.74~0.76秒で一定だった。

ノードの接続



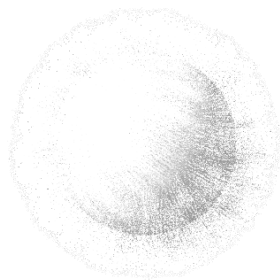
スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
skin	sphere	cols = 40 rows = 40 type = 2
normals	normal	(変更なし)
rest	rest	(変更なし)

ノード	タイプ	デフォルトから変更したパラメータ
roots	scatter::2.0	npts = 300
make_guides	attribwrangle	class = 0 snippet = int segs = 8; float len = 0.5; int n = npoints(0); for (int i = 0; i < n; i++) { vector p = point(0, "P", i); ...
hairgen	hairgen::2.0	hairprofile1interp = 5 hairprofile1pos = 0.17753121256828308 hairprofile1value = 1.0 hairprofile2interp = 5 hairprofile2pos = 0.8523035049438477 hairprofile2value = 1.0 hairprofile3interp = 5 hairprofile3pos = 1.0 influenceradius = 0.2 thickness = 0.008
out	merge	(変更なし)

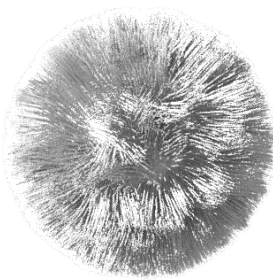
既定の太さと、上げた太さ

毛 12,516本。球も毛も同じで、thickness だけが違う。カメラは2枚とも同じ。Karma、9サンプル、600×600。面積はロゴの箱を除いて数えている。



Houdini

thickness 0.001（既定）。画の上で 0.103px。1ピクセルより細く、ザラついた点にしか見えない。



Houdini

thickness 0.008。画の上で 0.822px。毛の流れが読める。

thickness	画の上の太さ	触れたピクセル	平均の濃さ	時間
0.001（既定）	0.103 px	70,696	0.746	5.0秒
0.002	0.205 px	72,828	0.839	7.0秒
0.004	0.411 px	73,999	0.912	10.3秒
0.008	0.822 px	74,862	0.950	10.7秒

太さは指定どおりに出るか（毛を1本だけ、真横から）

毛の長さは 1.0。カメラの焦点距離と絞りから「1世界単位が何ピクセルか」を出し、512.95 px を正とした。ロゴの箱は測定から外してある。

thickness	予測(px)	実測(px)	実測 ÷ 予測	画に出た長さ（世界）
0.005	2.565	2.557	0.9969	1.00790
0.010	5.129	5.114	0.9969	1.01374
0.020	10.259	10.227	0.9969	1.02544
0.040	20.518	20.557	1.0019	1.04494

本数とレンダ時間

density だけを変えた。枠は毛の伸びる先まで含めて固定してあるので、どの density でもカメラは同じ。面積はロゴの箱を除く。

density	毛	時間	毛1本あたり	覆った面積
250	3,104	3.1秒	0.9841 ms	40,136
500	6,233	3.5秒	0.5629 ms	45,363
1000	12,516	4.7秒	0.3792 ms	52,736
2000	24,916	8.6秒	0.3470 ms	60,813
4000	49,977	13.1秒	0.2630 ms	67,276

OpenGL と Karma

同じ形・同じカメラ・同じ解像度で撮り比べた。

density	毛	OpenGL	Karma	倍率
250	3,104	0.74秒	3.1秒	4.1倍
1000	12,516	0.76秒	4.7秒	6.3倍
4000	49,977	0.75秒	13.1秒	17.4倍

わかったこと

- ・ width は直径だった。4通りで実測÷予測が 0.9969 / 0.9969 / 0.9969 / 1.0019。半径なら 2.0 になるはずで、そうはならない。3通りが 0.9969 でびたりとそろっている。
- ・ 最初の測定は 3〜4% ずれていた。原因はライセンスのロゴ。Apprentice のレンダには右下に Houdini のロゴが焼き込まれ、そこにも透明度がある。列ごとに透明度を足すやり方だと、ロゴのかかる列だけ値が水増しされる。ロゴの箱（行521〜566、列343〜566、透明度の合計 4163.3）を外したら 0.3% 以内に収まった。画を測るときは、画に写り込んでいるものを全部数え上げる。
- ・ 画に出る長さは、指定より少し長い。1.00790 / 1.01374 / 1.02544 / 1.04494。増えた分は太さとほぼ同じ（端が丸いので両側に半径ぶん出る）。太さを引いた残りは 0.0029〜0.0054 で、1.5〜2.8ピクセル分。輪郭のぼけにあたる。
- ・ 既定の thickness 0.001 は、この構図で 0.103ピクセルしかない。1ピクセルの1割。毛1本がピクセルの一部しか覆えないので、点が散らばったようにしか見えない。平均の濃さも 0.746 と薄い。
- ・ 太くすると絵が落ち着く。0.008（0.822px）で平均の濃さが 0.950 になり、毛の流れが読める。ただし時間は 5.0秒 → 10.7秒と倍になる。覆うピクセルが増えるぶん計算も増える。

- ・太さは世界の大きさに対する値である。0.001

が細すぎるのは、土台の球が半径1しかないから。数字そのものではなく「土台の何分の1か」で考える。

- ・レンダ時間は本数に比例しない。毛を16倍にしても時間は 4.2倍。毛1本あたりの時間は 0.9841ms → 0.2630ms と 3.7分の1になる。毛が増えるほど1本あたりは安くなる。

- ・覆う面積も比例しない。毛を16倍にしても、画の上で覆う面積は 40,136 → 67,276 と 1.68倍にしかない。手前の毛が奥の毛を隠すので、増やしても画は濃くなるだけで広がらない。

- ・OpenGL は本数によらず 0.74~0.76秒で一定だった。3,104本でも 49,977本でも変わらない。確認用に何度も撮るなら OpenGL、見せる絵は Karma、という使い分けでよい。

- ・Karma との差は本数とともに開く。4.1倍 → 6.3倍 → 17.4倍。毛のように数が増えるものでは、この差は効いてくる。

- ・毛1本を真横から撮るという測り方が鍵だった。毛玉のままでは毛が重なり、覆った面積から1本の太さを割り出せない。測りたいものだけを残す形に作り替える。

次にやること

- ・毛に色を付ける（根元と毛先で変える・clumpid ごとに変える）
- ・毛を動かす（風・重力）
- ・サンプル数と毛のザラつきの関係（実験018と同じ問いを毛で）

毛に色を付ける — 光を消して測ると、色は位置ぴったりに乗った

2026-09-24 | Houdini 21.0.700 | /obj/groom | ノード 9 / 接続 11

何を作ったか

実験045で毛を Karma に出せた。次は色を付ける。ところが毛が持っているアトリビュートは P と width だけだった。「この点は根元から何割の位置か」を示す数値は付いてこない。プリミティブごとに頂点の並び順から 0~1 を自分で振る。

色が位置どおりに乗っているかを確認するため、毛を1本だけ置いて根元を赤・毛先を青にし、真横から撮った。画の列ごとに青の割合を測ると、光を当てた状態では $0.6877 \times \text{位置} + 0.1748$

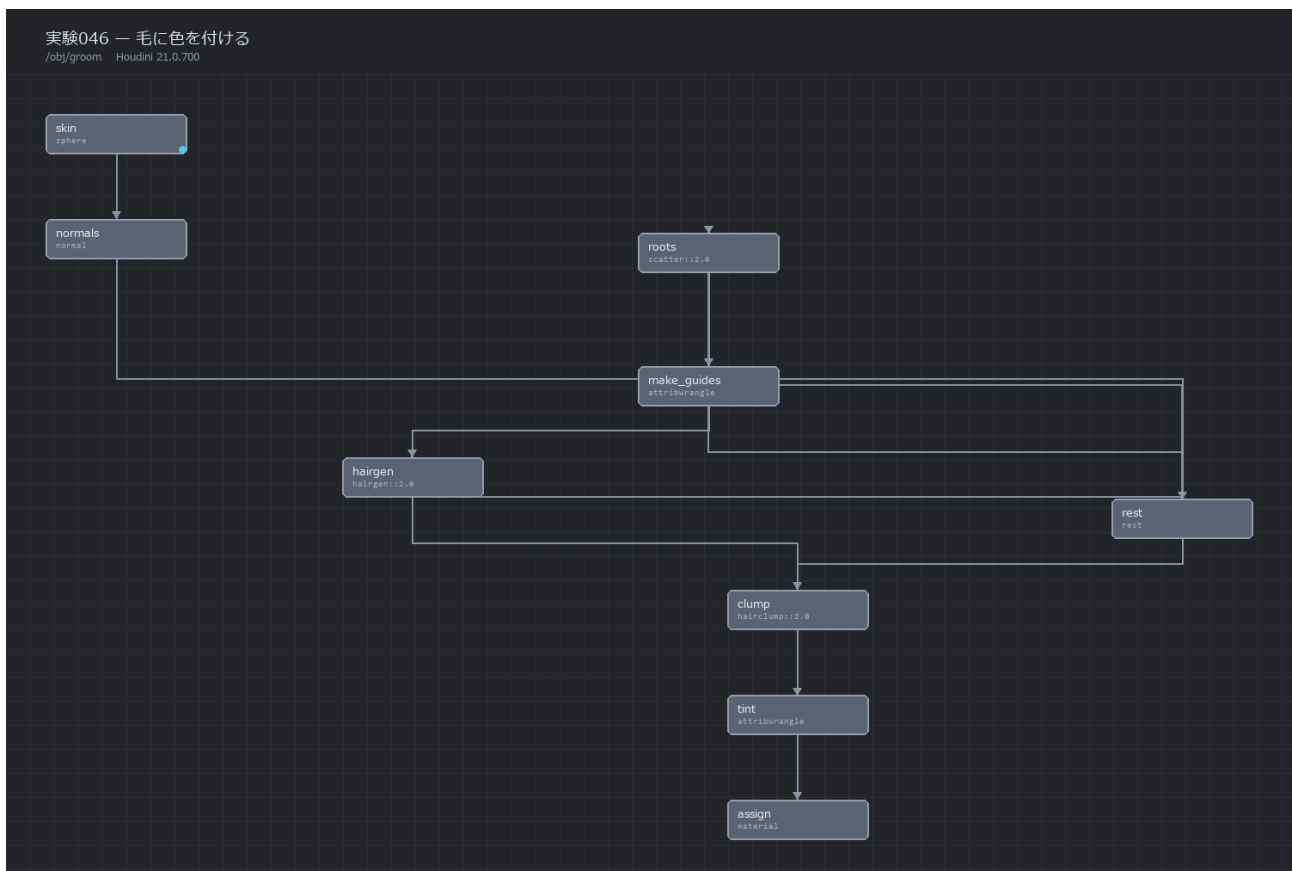
という直線になる。比例はしているが、1:1

ではない。周囲の光が赤にも青にも同じだけ乗るためで、この見立てが正しければ「傾き + 2×切片 = 1」になるはず。実測は 1.0373 だった。

光の影響を外すため、色を発光として出すと $1.0123 \times \text{位置} - 0.0064$ 、 $R^2 = 0.99991$ 、直線からの最大のずれ 0.0037。青の割合は位置そのものになった。

ついでに width も測った。根元から 1/4 までは 0.008 のままで、そこから細くなり、毛先でちょうど 0 になる。パラメータにある hairprofile というカーブのとおりに先細りしている。ばらつきは 12,516本すべてで 0.00000000 だった。

ノードの接続

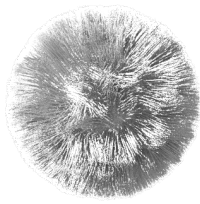


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
skin	sphere	cols = 40 rows = 40 type = 2
normals	normal	(変更なし)
rest	rest	(変更なし)
roots	scatter::2.0	npts = 300
make_guides	attribwrangle	class = 0 snippet = int segs = 8; float len = 0.5; int n = npoints(0); for (int i = 0; i < n; i++) { vector p = point(0, "P", i); ...
hairgen	hairgen::2.0	hairprofile1interp = 5 hairprofile1pos = 0.17753121256828308 hairprofile1value = 1.0 hairprofile2interp = 5 hairprofile2pos = 0.8523035049438477 hairprofile2value = 1.0 hairprofile3interp = 5 hairprofile3pos = 1.0 influenceradius = 0.2 thickness = 0.008
clump	hairclump::2.0	blendremap2pos = 1.0 blendremap2value = 1.0 clumpprofile1interp = 5 clumpprofile1value = 1.0 clumpprofile2interp = 5 clumpprofile2pos = 0.14254385232925415 clumpprofile2value = 0.48605579137802124 clumpprofile3interp = 5 clumpprofile3pos = 0.5679824352264404 clumpprofile3value = 0.11952191591262817 clumpprofile4interp = 5 clumpprofile4pos = 1.0 ...ほか 27 件
tint	attribwrangle	class = 1 snippet = int pts[] = primpoints(0, @primnum); int n = len(pts); for (int i = 0; i < n; i++) { float u = (n > 1) ? float(i...
assign	material	shop_materialpath1 = /mat/hair_shader

色を付けない／根元から毛先へ／束ごと

毛 12,516本。形もカメラも同じで、色の付け方だけが違う。Karma、9サンプル、600×600。



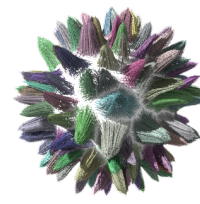
Houdini

色を付けない。彩度のあるピクセルは 0.0%。



Houdini

根元を赤、毛先を青にした。彩度のあ
るピクセルは 90.1%。



Houdini

束ごとに色を変えた。144個の束が目
で数えられる。

条件	毛	時間	色のあるピクセル	彩度の平均
色なし	12,516	8.0秒	0.0%	0.0000
根元→毛先	12,516	8.0秒	90.1%	0.5745
束ごと	12,516	8.1秒	56.4%	0.1808

色は位置どおりに乗るか（毛を1本だけ、真横から）

根元を赤(1,0,0)、毛先を青(0,0,1)にして、その間を位置で混ぜた。画の列ごとに、透明度で重みを付けて赤と青の平均を出し、sRGB

を線形に戻してから青の割合を求めている。17か所を測って直線をあてはめた（表は4か所おき）。

位置	赤(sRGB)	青(sRGB)	青の割合	直線との差	光
0.10	217.8	139.7	0.2718	+0.0282	当てる
0.30	203.1	160.5	0.3718	-0.0093	当てる
0.50	178.7	179.0	0.5009	-0.0177	当てる
0.70	149.7	199.0	0.6530	-0.0032	当てる
0.90	108.3	214.5	0.8176	+0.0239	当てる
0.10	246.1	86.5	0.0926	-0.0023	発光
0.30	220.9	150.3	0.2976	+0.0003	発光
0.50	189.4	190.3	0.5028	+0.0031	発光
0.70	151.3	220.6	0.6987	-0.0035	発光
0.90	85.7	245.3	0.9084	+0.0037	発光

あてはめた直線

周囲の光は赤にも青にも同じだけ乗るので、青の割合は (位置×k + a) ÷ (k + 2a)

という位置の1次式になるはず。この形なら「傾き + 2×切片 = 1」が成り立つ。

光	傾き	切片	R ²	直線からの最大のずれ	傾き + 2×切片
当てる	0.6877	0.1748	0.99142	0.0282	1.0373
発光	1.0123	-0.0064	0.99991	0.0037	0.9996

width は根元から先まで一定か

thickness 0.008 で 2,500本ぶんを平均した。位置は毛の中の点の並び（9点＝8区間）。

位置	width の平均	根元との比	ばらつき
0.000	0.008000	1.0000	0.00000000
0.125	0.008000	1.0000	0.00000000
0.250	0.008000	1.0000	0.00000000
0.375	0.007987	0.9984	0.00000000
0.500	0.007865	0.9831	0.00000000
0.625	0.007536	0.9419	0.00000000
0.750	0.006802	0.8502	0.00000000
0.875	0.004981	0.6227	0.00000000
1.000	0.000000	0.0000	0.00000000

わかったこと

- ・毛が持っているのは P と width だけだった。「根元から何割の位置か」を示す数値は付いてこない。プリミティブごとに primpoints で点を並べ、順番から 0～1 を自分で振る。頂点にもディテールにも何も入っていない。
- ・光を当てたまま測ると、色は比例するが 1:1 にはならない。青の割合 = $0.6877 \times \text{位置} + 0.1748$ 。周囲の光が赤にも青にも同じだけ乗るので、割合が 0.5 側へ寄る。
- ・その見立ては検算できる。周囲の光を a、色そのものの強さを k とすると、青の割合は $(\text{位置} \times k + a) \div (k + 2a)$ 。この形なら 傾き + 2×切片 = 1 になる。実測は 1.0373（3.7% のずれ）。
- ・発光にして光の影響を外すと、位置そのものになった。 $1.0123 \times \text{位置} - 0.0064$ 、 $R^2 = 0.99991$ 、直線からの最大のずれ 0.0037。傾き + 2×切片 も 0.9996。色を測りたいときは、光を当てずに測る。
- ・width は一定ではなく、先細りしていた。根元から 1/4 までは 0.008 のままで、そこから落ちて毛先でちょうど 0。hairgen のパラメータにある hairprofile というカーブのとおりに変えられている。
- ・先細りは 12,516本すべてで同じだった。どの位置でもばらつきが 0.00000000。毛ごとの個性ではなく、全部に同じカーブが掛かっている。
- ・色を付けてもレンダ時間はほとんど変わらない。8.0秒 / 8.0秒 / 8.1秒。色は形の複雑さを増やさない。
- ・束ごとに色を変えると、束が目で見えらる。実験043で測った144個の束が、そのまま房として見える。数字で出した結果を、見た目で確かめられる形にしておく。
- ・測り方について。実験045で見つけた「Apprentice のロゴが測定に混ざる」問題は、この実験でも最初に出た。光を当てた側の R^2 が 0.94 にしかならず、毛の右半分だけ値が乱れていた。ロゴの箱を外したら 0.99142 になった。同じ罫は、画を測るかぎり毎回出る。

次にやること

- ・毛を動かす（風・重力）

- ・ サンプル数と毛のザラつきの関係（実験018と同じ問いを毛で）
- ・ 毛を土台のアニメーションに付いてこさせる

ターンテーブルで見せる ― 形は物を回してもカメラを回しても同じ。変わるのは光だけ

2026-09-24

何を作ったか

作ったものを1周見せるターンテーブルには、物を回すやり方と、カメラを回すやり方がある。同じ絵になるのか、どちらが速いのかを測った。見せる物は、実験083で作った立体文字「SP」。

まず条件をそろえるのに2回つまずいた。1回目は、カメラの高さの比を物を回すときと変えてしまい、比べられなかった。2回目は、文字を原点のまわりに置くずらし量を当て推量で入れていて、回すと画の中で位置がずれた。外接箱の真ん中を測ってずらし直すと、ようやく比べられる形になった。

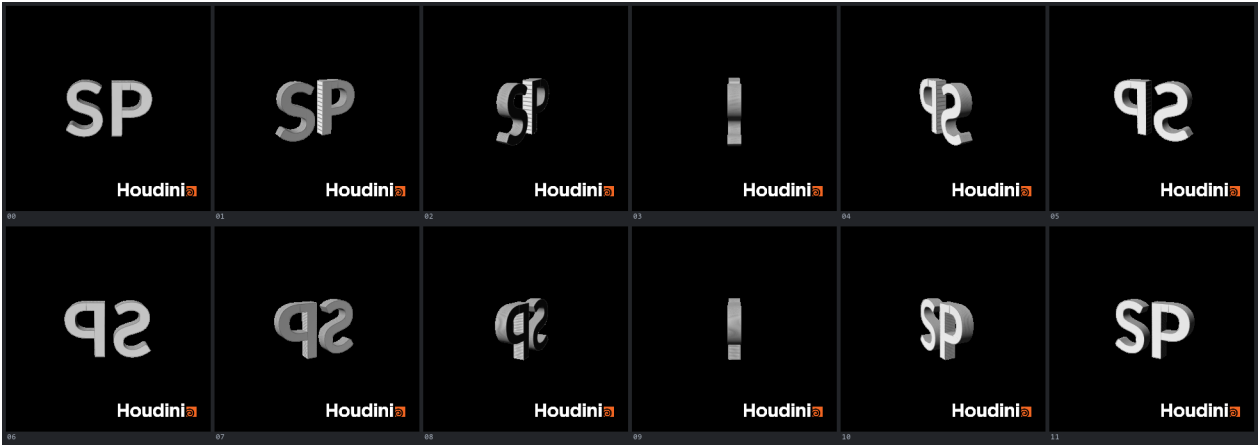
そろえて撮ると、形はぴったり同じだった。シルエット（画面に占める割合）は、4コマとも 7.61% / 7.61%、9.28% / 9.29%、9.83% / 9.83%、9.04% / 9.05%。

違うのは明るさだった。物を回すと、写っている部分の明るさの平均は 182.6 / 176.2 / 186.4 / 178.2 とほぼ一定。カメラを回すと 186.5 / 132.8 / 51.1 / 60.2 と、裏へ回り込むほど暗くなる。ライトは世界に固定されているので、物を回せば当たり方は変わらず、カメラを回せば影の側へ入っていく。

時間は軽い。320 × 320 で1コマ 13ms、480 で 18ms、720 で 29～30ms。24コマの1周でも 0.3～0.7秒。GIF にすると、320 × 320 ・ 24コマで 128,996バイト（元の PNG 合計 272,287バイトの半分弱）。

1周を12コマで（物を回した場合）

320 × 320。0コマ目から30度ずつ回している。3コマ目と9コマ目が真横で、文字が線になる。



12コマ。1周のどこで何が見えるかが、これで分かる。

A. 物を回す / カメラを回す

同じ角度になるよう、カメラは物と逆向きに同じ角度だけ回した。高さの比もそろえてある。「明るさ」は、写っている画素（明るさ5より上）の平均。

コマ	画素の平均の差	シルエット（物を回す）	シルエット（カメラを回す）	明るさ（物を回す）	明るさ（カメラを回す）
1（0度）	0.296	7.61%	7.61%	182.6	186.5
7（90度）	8.290	9.28%	9.29%	176.2	132.8

コマ	画素の平均の差	シルエット（物を回す）	シルエット（カメラを回す）	明るさ（物を回す）	明るさ（カメラを回す）
13（180度）	13.495	9.83%	9.83%	186.4	51.1
19（270度）	12.109	9.04%	9.05%	178.2	60.2

B. 1コマあたりの時間（OpenGL）

320 × 320 の12コマだけ 75ms

と大きいのは、1回目に下ごしらえの時間が乗るため（実験034・061と同じ）。

解像度	コマ数	合計	1コマ	PNG 1枚
320 × 320	12	0.90秒	75 ms	12,605 バイト
320 × 320	24	0.31秒	13 ms	11,726 バイト
480 × 480	12	0.22秒	18 ms	23,451 バイト
480 × 480	24	0.43秒	18 ms	21,270 バイト
720 × 720	12	0.36秒	30 ms	39,476 バイト
720 × 720	24	0.70秒	29 ms	34,822 バイト

C. GIF にしたときの大きさ

PNG の連番をそのまま GIF にまとめた（1コマ 90ms）。

解像度	コマ数	GIF	元の PNG 合計	GIF ÷ PNG
320 × 320	12	64,048 バイト	133,978 バイト	0.48
320 × 320	24	128,996 バイト	272,287 バイト	0.47
480 × 480	24	242,054 バイト	506,408 バイト	0.48

わかったこと

- ・形はどちらのやり方でも同じ。シルエットの割合が 7.61% / 7.61%、9.83% / 9.83% と、0.01ポイントまで一致した。
- ・違うのは光の当たり方。物を回すと明るさは 176～186 でほぼ一定、カメラを回すと 186 → 51 と暗くなる。ライトは世界に固定されているからで、カメラだけ回すと影の側へ入る。
- ・見せるだけなら、物を回すほうが安定する。カメラを回したいなら、ライトもカメラと一緒に回す（カメラに付ける）必要がある。
- ・時間は気にしなくてよい。OpenGL で1コマ 13～30ms、24コマの1周でも 0.3～0.7秒。解像度を 320 → 720（画素で 5倍）にしても、1コマ 13ms → 29ms（2.2倍）。
- ・GIF は PNG 合計の半分弱。0.47～0.48倍。320 × 320・24コマで 129KB なので、ページに置ける大きさ。
- ・物を回すときは、外接箱の真ん中を測ってから原点に寄せる。ずらす量を当て推量にしたら、回転のたびに画の中で位置がずれ、シルエットの割合が 14.11% と 5.37% のように食い違った。
- ・測り方について。形と明るさを分けて測ったことで、食い違いの理由が1回で分かった。「写っている割合」だけを見ていたときは、形が違うのか暗いだけなのか区別できなかった。

次にやること

- ・ ライトをカメラに付けて回すと、明るさは一定になるか
- ・ 光らせる（発光する材質）
- ・ 振り返り点検（061～090）

文字を光らせる — 明るさは頭打ち、こぼれる光と時間だけが増える

2026-09-24

何を作ったか

立体文字を床の上に置き、文字の材質に発光（Emission）を入れて Karma で撮った。ライトはドーム 0.12・キー 0.05 だけにして、発光の効き方だけを見る。

Emission Intensity を 0 / 1 / 4 / 16 / 64 と上げると、文字のあたりの明るさは 43.54 → 72.00 → 85.42 → 98.64 → 100.39。16 から 64 へ4倍にしても 1.75 しか上がらない。画素は 255 で頭打ちになるので、強くするほど「白い面積」が増えるだけになる（白飛びは 0.85% → 4.74%）。

増え続けるのは、まわりへこぼれる光と時間だった。床のあたりの明るさは 96.89 → 143.07。撮る時間は 3.79秒 → 12.35秒で、3.3倍になる。

Emission Illuminates Objects を切ると、床は光らなくなった。Emission 16 で、床の明るさが 126.95 から 96.89 へ下がる。この 96.89 は、発光を 0 にしたときの床の明るさと6桁まで同じ。切ると「光って見えるが、まわりは照らさない」状態になる。そのぶん速く、10.27秒が 3.40秒（3.0倍）になった。

発光の強さ

Karma 640 × 400 ・ サンプル24。3枚とも同じカメラ・同じライト。



Emission 0。ドームの弱い光だけで、文字は暗い。

Emission 4。文字が光り、床にも色乗る。

Emission 64。文字は白く飛び、床のこぼれ光が広がる。

A. Emission Intensity

「文字」は画面の真ん中あたり、「床」は下の帯の平均（0～255）。白飛びは 250 を超えた画素の割合。

Emission	文字の明るさ	床の明るさ	白飛び	撮る時間
0	43.54	96.89	0.85%	3.79秒
1	72.00	99.44	0.89%	5.48秒
4	85.42	106.27	0.96%	9.35秒
16	98.64	126.95	4.42%	11.83秒
64	100.39	143.07	4.74%	12.35秒

B. Emission Illuminates Objects (Emission 16)

切ったときの床の明るさ 96.89 は、発光 0 のときと同じ値。

emitillum	文字の明るさ	床の明るさ	白飛び	撮る時間
入り（既定）	98.64	126.95	4.42%	10.27秒
切り	83.55	96.89	4.27%	3.40秒

わかったこと

- ・文字の明るさは頭打ちになる。Emission 16 → 64 で 98.64 → 100.39（+1.8%）。画素が 255 で止まるので、それ以上は白い面積が増えるだけ（白飛び 4.42% → 4.74%）。
- ・増え続けるのは、こぼれる光。床の明るさは 96.89（0）→ 126.95（16）→ 143.07（64）。「もっと光らせたい」ときに効くのは、文字ではなく周りの明るさ。
- ・発光は重い。同じ絵で 3.79秒（0）→ 12.35秒（64）と 3.3倍。ライトを足したのと同じように、光源が1つ増えた扱いになる。
- ・Emission Illuminates Objects を切ると、まわりを照らさなくなる。床の明るさは 126.95 → 96.89 で、発光 0 のときと同じ値に戻った。文字自体は光ったまま（83.55）。時間は 10.27秒 → 3.40秒と3.0倍速い。
- ・実用の結論。看板の文字のように「光って見えればよい」なら、emitillum を切って Emission を 4～16 に。まわりを照らしたいときだけ入れる。そのときは時間が3倍になることを見込む。
- ・測り方について。文字と床を別々の帯として測った。1枚の平均だけでは、文字が明るいのか床が明るいのか分からない。

次にやること

- ・落ち葉を地面に積もらせる
- ・発光の色を変えると、こぼれる光の色はどうか
- ・振り返り点検（061～090）

箱 100 個を USD にすると、パックしなければメッシュ 1 つ、Point Instancer なら 6 プリム、Xforms なら 202 プリム

2026-09-24

何を作ったか

SOP で箱を 100 個並べ（copytopoints）、パックしたもの（Pack and Instance）としないものを LOP の sopimport で読み込み、できた USD のプリムを種類ごとに数えた（SOP では、パックしない方が面 600 枚、パックした方がパック 100 個）。

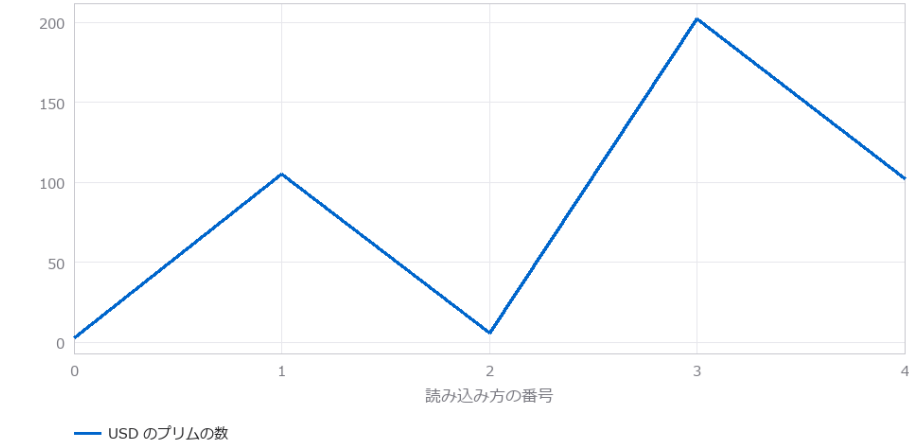
****パックしないと、100 個の箱は Mesh 1 つにまとまる****（プリムは全部で 3）。1 つずつ動かしたり選んだりできない。

****パックしたものは、Packed Primitives の扱いでまったく違う形になる。***既定の Native Instances では Xform 102・インスタンス 100・元の形（プロトタイプ）1。Point Instancer では PointInstancer が 1 つで、プリムは全部で 6。Xforms では Xform と Mesh が 100 個ずつ（202）で、同じ形を 100 回持つ。Unpack では Mesh 100 個（102）。

****読み込みの時間はどれも数ミリ秒。***ただし、そのセッションで最初の LOP だけは 21 秒かかった（USD の立ち上げ）。時間をはかるときは、先に1回読み込んでおく必要がある。

読み込み方と USD のプリム

箱 100 個を sopimport: 0 パックしない / 1 Native Instances / 2 Point Instance



Xforms（3）が一番多く、Point Instancer（2）が一番少ない。

SOP	Packed Primitives	プリムの数	種類	インスタンス	プロトタイプ	秒
パックしない	パックしない	3	Xform 1・Mesh 1・HoudiniLayerInfo 1	0	0	0.0034
パックした	Native Instances（既定）	105	Xform 102・Scope 1・Mesh 1・HoudiniLayerInfo 1	100	1	0.0048

SOP	Packed Primitives	プリムの数	種類	インスタンス	プロトタイプ	秒
パックした	Point Instancer	6	Xform 2・PointInstancer 1・Scope 1・Mesh 1・HoudiniLayerInfo 1	0	0	0.0037
パックした	Xforms	202	Xform 101・Mesh 100・HoudiniLayerInfo 1	0	0	0.0054
パックした	Unpack	102	Xform 1・Mesh 100・HoudiniLayerInfo 1	0	0	0.0052

わかったこと

- ・数を増やすなら Point Instancer。100 個でもプリムは 6。
- ・1 個ずつを別のプリムとして選んだり動かしたりしたいなら Native Instances か Xforms。その分ファイルは大きくなる（1 万個で Point Instancer の 4.6～5.1 倍、実験190）。
- ・パックせずに読み込むと、全部が1つのメッシュになる。

次にやること

- ・1 万個にしたときの読み込み時間とファイルの大きさ
- ・Point Instancer をレンダーしたときの時間

箱 1 万個の USD — Point Instancer は 0.32 MB・8 ミリ秒、Native Instances は 4.6 倍、Unpack は 7.4 倍の大きさ

2026-09-24

何を作ったか

実験189 の続き。箱を $100 \times 100 = 1$ 万個並べて sopimport で USD にし、読み込み時間と、ステージを .usdc で書き出したファイルの大きさを比べた（USD の立ち上げは先に済ませた）。

****Point Instancer がいちばん軽く、速い。*プリム 6、ファイル 0.32 MB、読み込み 8 ミリ秒。1 万個の位置と向きを、1 つのプリムの配列として持つ。**

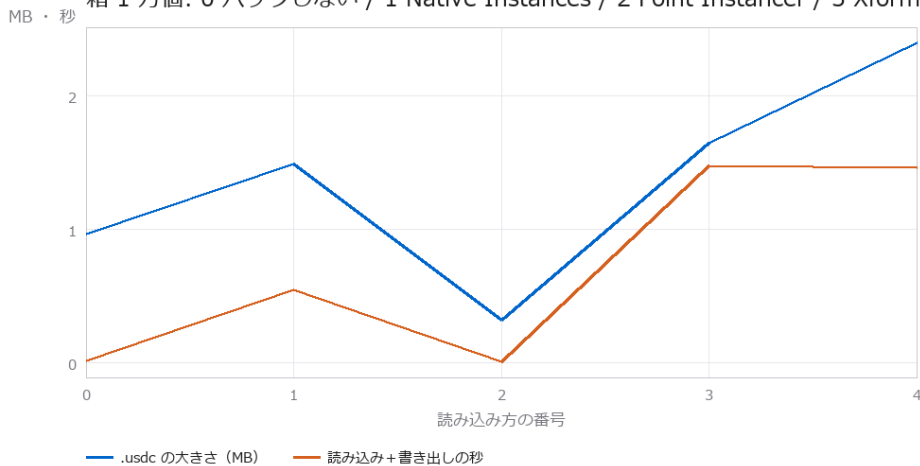
****Native Instances はプリムが 1 万を超え、ファイルは 4.6 倍**（1.49 MB、読み込み 123 ミリ秒、書き出し 0.43 秒）。Xforms は 1.64 MB（5.1 倍、書き出し 1.22 秒）、Unpack は 2.40 MB（7.4 倍）でいちばん大きい。**

****Xforms（形を 1 万回持つ）は、Unpack より小さかった。*同じ形のメッシュを、書き出すときにまとめている可能性があるが、確かめていない。実験189 で「Xforms は重くなる」と書いたのは、Native Instances や Point Instancer と比べたときの話で、Unpack より軽い。**

****パックスせずに読むと、1 万個の箱が Mesh 1 つになり 0.96 MB。*1 個ずつ扱えない代わりに、ファイルは Native Instances より小さい。**

読み込み方と、プリム・時間・ファイル

箱 1 万個: 0 パックしない / 1 Native Instances / 2 Point Instancer / 3 Xforms /



Point Instancer（2）が大きさも時間もいちばん下。

SOP	Packed Primitives	プリム	読み込みの秒	書き出しの秒	.usdc の大きさ	Point Instancer の何倍
パックスしない	パックスしない	3	0.0102	0.0031	0.96 MB	3.0
パックスした	Native Instances (既定)	10,005	0.1229	0.4261	1.49 MB	4.6

SOP	Packed Primitives	プリム	読み込みの秒	書き出しの秒	.usdc の大きさ	Point Instancer の何倍
パックした	Point Instancer	6	0.0084	0.0021	0.32 MB	1.0
パックした	Xforms	20,002	0.2576	1.2195	1.64 MB	5.1
パックした	Unpack	10,002	0.3963	1.0674	2.40 MB	7.4

わかったこと

- たくさん並べるなら Point Instancer。1 万個で 0.32 MB。
- Native Instances は、1 個ずつがプリムになる分だけ重い。1 万個で 1.5 MB、書き出し 0.4 秒。
- Unpack がいちばん大きい。パックを USD に渡すとき、わざわざ解く必要はない。

次にやること

- Point Instancer と Native Instances の Karma でのレンダー時間
- 10 万個にしたときの伸び方